

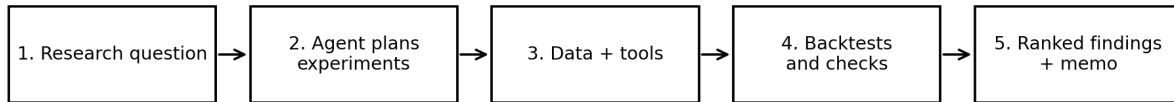
Alpha Research Agent - Public Guide

An LLM-driven quantitative research prototype with deterministic backtesting, research controls, and an optional FastAPI interface.

What it does end to end

A user supplies a plain-English quantitative research question. The agent translates it into a small family of momentum experiments, calls deterministic Python tools to run the backtests, compares the evidence, and writes a research memo. The language model decides what to test; ordinary Python calculates the numbers.

End-to-end workflow



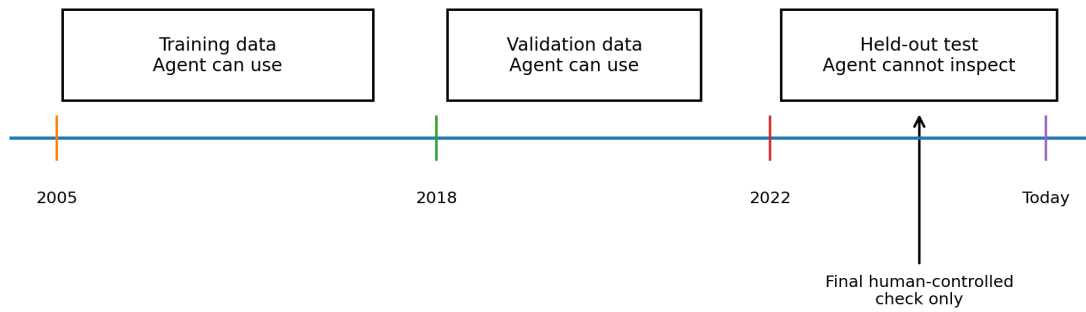
The LLM chooses what to test; deterministic Python computes the results.

Layer	Current implementation
Market data	Yahoo Finance through yfinance; daily adjusted market prices
Universe	9 long-history U.S. sector ETFs
Agent framework	OpenAI Agents SDK
Quant tools	pandas, numpy, scipy and a custom backtesting engine
Memory	Local JSONL experiment history and saved Markdown memos
HTTP interface	FastAPI service with Swagger documentation at /docs
Final test	Human-controlled finalize.py; not exposed to the agent or API

Research controls

The project is intentionally restrictive. It shifts weights before earning returns to reduce look-ahead bias, applies turnover-based transaction costs, limits each research session to eight experiments, and keeps the final test period sealed while the agent is choosing candidate specifications.

Why the test set is sealed



This helps prevent the agent from tuning itself to future results.

The backtester reports both gross and net performance, annualized return and volatility, Sharpe ratio, drawdown, turnover, hit rate, an IID t-statistic, and a simple HAC/Newey-West-style t-statistic.

FastAPI interface

The same research engine can be launched as a local HTTP service. This is useful for a future web frontend, notebook integration, or another program that wants to call the agent. The API is intentionally narrower than the full CLI: it exposes research and inspection endpoints, but not the held-out final test.

Method	Endpoint	Purpose
GET	/	Basic service metadata
GET	/health	Check whether the API process is alive
GET	/dataset	Inspect the available dataset and research split
GET	/experiments	Read experiment history
POST	/research	Run a research session and return the final memo

Launch locally

After installing dependencies and configuring the local `.env` file, run `fastapi dev`. Then open `http://127.0.0.1:8000/docs` for interactive Swagger documentation. The repository includes a FastAPI entry point in `pyproject.toml`.

The absence of a `/finalize` endpoint is deliberate. A web client should not be able to repeatedly query the held-out test set while the strategy is still being selected.

Command-line workflow

- `python run.py --reset` - run the default research question.
- `python run.py --reset --question "..."` - run a custom question.
- `pytest -q` - run the synthetic and API test suite.
- `python finalize.py 3` - open the held-out test only after choosing experiment 3.

What is saved

Downloaded prices are cached under `data/`. Experiment records are written to `results/experiments.jsonl`, and the latest research memo is saved to `results/latest_memo.md` with timestamped copies in `results/memos/`. These generated artifacts are ignored by Git.

Scope and limitations

This is an educational/research prototype rather than an institutional trading system. The ETF universe is small, Yahoo Finance is not institutional-grade data, transaction costs are simplified, and repeated human prompting can still introduce selection bias. The purpose is to demonstrate auditable agent-driven research, not to promise profitable trading signals.